

数据融合与智能分析实验报告 3

班级	物联网 2201	学号	223428010103
专业	物联网工程	姓名	刘渤
实验表现成绩		实验成绩	
报告成绩			
指导教师	郭振洲		

一、实验题目：数据分析综合实验

二、实验地点：雨课堂

三、实验目的：

- 1 理解 CDNOW_master 数据含义；
- 2 理解时间序列函数的转换和相关应用；
- 3 能够应用 groupby,透视表等命令实现对数据的综合查询；

四、实验内容（完成题目代码及运行结果，代码注释中补充个人信息，以示区别）

1. 数据加载 字段含义： user_id:用户 ID order_dt:购买日期 order_product:购买产品的数量 order_amount:购买金额

（1）观察数据

```
#表头为用户ID，购买日期，购买数量，购买金额
columns=['user_id','order_dt','order_products','order_amount']
df = pd.read_csv('CDNOW_master.txt', names=columns, sep='\\s+')

# 223428010210 陈梓欣
# 观察数据
print(df)
```

	user_id	order_dt	order_products	order_amount
0	1	19970101	1	11.77
1	2	19970112	1	12.00
2	2	19970112	5	77.00
3	3	19970102	2	20.76
4	3	19970330	2	20.76
...	...	...	...	...
69654	23568	19970405	4	83.74
69655	23568	19970422	1	14.99
69656	23569	19970325	2	25.74
69657	23570	19970325	3	51.12
69658	23570	19970326	2	42.96

[69659 rows x 4 columns]

(2) 查看数据的数据类型

```
# 223428010210 陈梓欣
print(df.dtypes)
```

```
user_id      int64
order_dt     int64
order_products int64
order_amount float64
dtype: object
```

(3) 数据中是否存储在缺失值

```
# 223428010210 陈梓欣
# 数据中是否存在缺失值
print(df.isnull().sum())
```

```
user_id      0
order_dt     0
order_products 0
order_amount 0
dtype: int64
```

(4) 将 order_dt 转换成时间类型

```
# 223428010210 陈梓欣
# 时间数据类型转换
df['order_dt'] = pd.to_datetime(df['order_dt'], format='%Y%m%d')
print(df.head())
```

	user_id	order_dt	order_products	order_amount
0	1	1997-01-01	1	11.77
1	2	1997-01-12	1	12.00
2	2	1997-01-12	5	77.00
3	3	1997-01-02	2	20.76
4	3	1997-03-30	2	20.76

(5) 查看数据的统计描述

```
# 223428010210 陈梓欣
# 查看数据的统计描述
print(df.describe())
```

	user_id	order_dt	order_products	\
count	69659.000000	69659	69659.000000	
mean	11470.854592	1997-07-02 22:36:51.401254656	2.410040	
min	1.000000	1997-01-01 00:00:00	1.000000	
25%	5506.000000	1997-02-22 00:00:00	1.000000	
50%	11410.000000	1997-04-24 00:00:00	2.000000	
75%	17273.000000	1997-11-07 00:00:00	3.000000	
max	23570.000000	1998-06-30 00:00:00	99.000000	
std	6819.904848	NaN	2.333924	

	order_amount
count	69659.000000
mean	35.893648
min	0.000000
25%	14.490000
50%	25.980000
75%	43.700000
max	1286.010000
std	36.281942

(6) 计算所有用户购买商品的平均数量

```
# 223428010210 陈梓欣
# 计算所有用户购买商品的平均数量
avg_quantity = df['order_products'].mean()
print("avg_quantity: ", avg_quantity)
```

avg_quantity: 2.41004033936749

(7) 计算所有用户购买商品的平均花费

```
# 223428010210 陈梓欣
# 计算所有用户购买商品的平均花费
avg_amount = df['order_amount'].mean()
print("avg_amount: ", avg_amount)
```

avg_amount: 35.89364805696321

(8) 在源数据中添加一列表示月份:astype('datetime64[M]').

```
# 223428010210 陈梓欣
# 在源数据中添加一列表示月份, 默认是每月的第1天
df['month'] = df.order_dt.values.astype('datetime64[M]')
print(df.head())
```

	user_id	order_dt	order_products	order_amount	month
0	1	1997-01-01	1	11.77	1997-01-01
1	2	1997-01-12	1	12.00	1997-01-01
2	2	1997-01-12	5	77.00	1997-01-01
3	3	1997-01-02	2	20.76	1997-01-01
4	3	1997-03-30	2	20.76	1997-03-01

2 按月数据分析

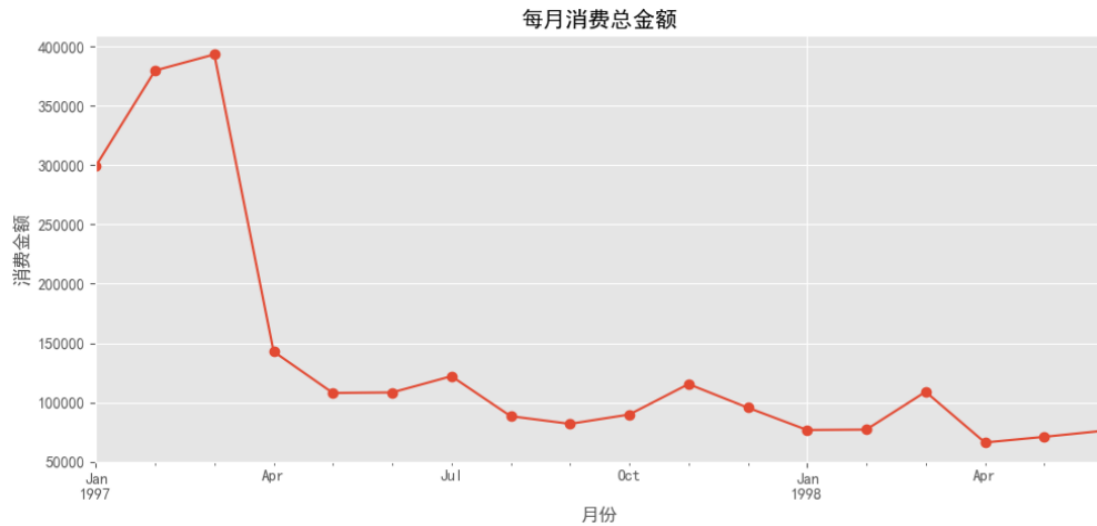
(1) 每月消费的总金额, 绘制曲线图展示

```
# 223428010210 陈梓欣
# 每月消费的总金额
monthly_amount = df['order_amount'].groupby(df['month']).sum()
print(monthly_amount)
```

```
month
1997-01-01    299060.17
1997-02-01    379590.03
1997-03-01    393155.27
1997-04-01    142824.49
1997-05-01    107933.30
1997-06-01    108395.87
1997-07-01    122078.88
1997-08-01     88367.69
1997-09-01     81948.80
1997-10-01     89780.77
1997-11-01    115448.64
1997-12-01     95577.35
1998-01-01     76756.78
1998-02-01     77096.96
1998-03-01    108970.15
1998-04-01     66231.52
1998-05-01     70989.66
1998-06-01     76109.30
Name: order_amount, dtype: float64
```

```
#223428010210 陈梓欣
import matplotlib.pyplot as plt

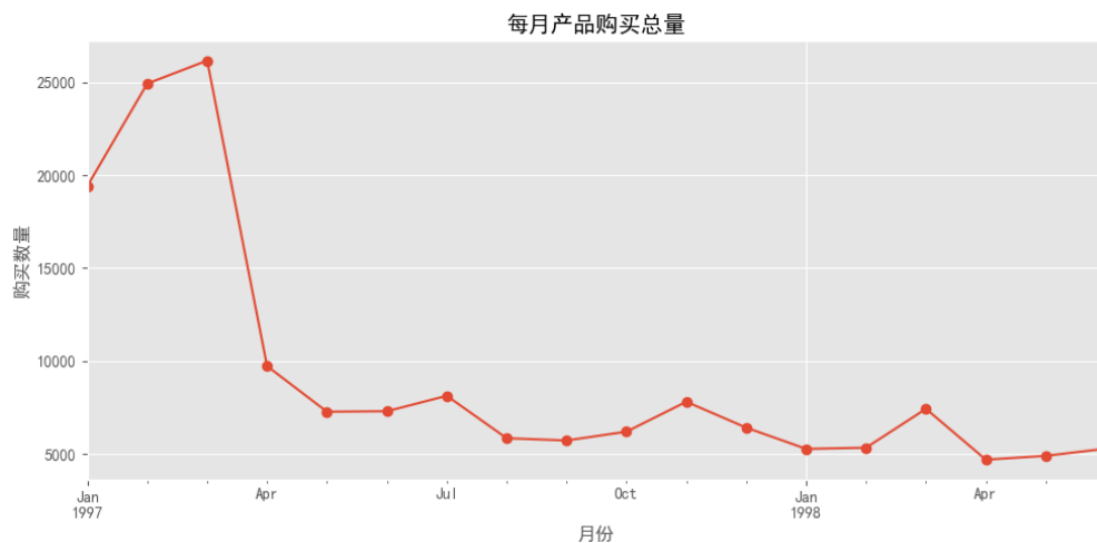
# 绘制每月消费金额变化趋势图
monthly_amount.plot(title='每月消费总金额', figsize=(10, 5), marker='o')
plt.xlabel('月份')
plt.ylabel('消费金额')
plt.grid(True)
plt.tight_layout()
plt.show()
```



(2) 所有用户每月的产品购买量

```
# 223428010210 陈梓欣
# 所有用户每月的产品购买量
monthly_products = df['order_products'].groupby(df['month']).sum()

# 绘图
monthly_products.plot(title='每月产品购买总量', figsize=(10, 5), marker='o')
plt.xlabel('月份')
plt.ylabel('购买数量')
plt.grid(True)
plt.tight_layout()
plt.show()
```



(3) 所有用户每月的消费总次数

```
# 223428010210 陈梓欣
# 每月消费总次数
monthly_transaction_count = df.groupby('month').size()

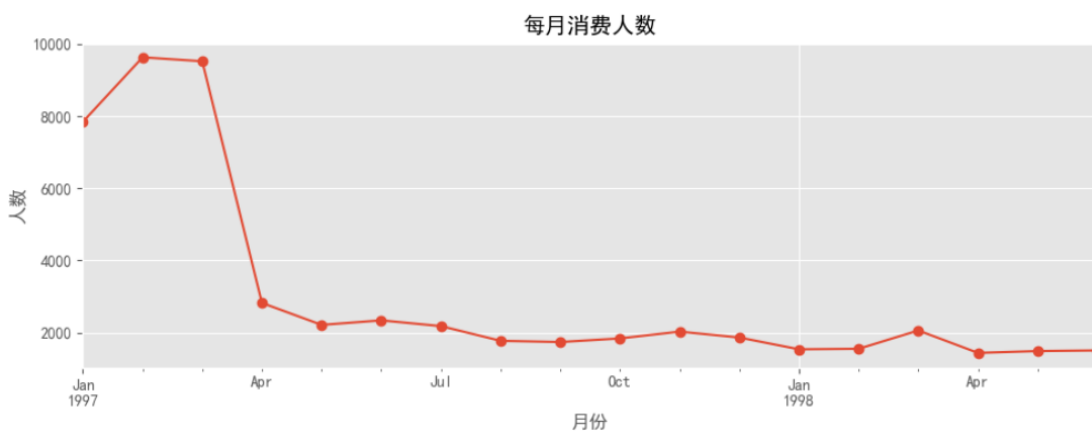
# 折线图: 每月消费总次数
monthly_transaction_count.plot(title='每月消费总次数', figsize=(10, 4), marker='o')
plt.xlabel('月份')
plt.ylabel('订单数 (次)')
plt.grid(True)
plt.tight_layout()
plt.show()
```



(4) 统计每月的消费人数

```
# 223428010210 陈梓欣
# 每月消费人数
monthly_unique_users = df.groupby('month')['user_id'].nunique()

# 折线图: 每月消费人数
monthly_unique_users.plot(title='每月消费人数', figsize=(10, 4), marker='o')
plt.xlabel('月份')
plt.ylabel('人数')
plt.grid(True)
plt.tight_layout()
plt.show()
```



3 用户个体消费数据分析

(1) 用户消费总金额和消费总次数的统计描述

```
# 223428010210 陈梓欣
# 消费金额和消费次数的统计描述
print("用户消费总金额描述: ")
print(user_group['order_amount'].describe())

print("\n用户消费总次数描述: ")
print(user_group['order_count'].describe())
```

用户消费总金额描述:

count	23570.000000
mean	106.080426
std	240.925195
min	0.000000
25%	19.970000
50%	43.395000
75%	106.475000
max	13990.930000

Name: order_amount, dtype: float64

用户消费总次数描述:

count	23570.000000
mean	2.955409
std	4.736558
min	1.000000
25%	1.000000
50%	1.000000
75%	3.000000
max	217.000000

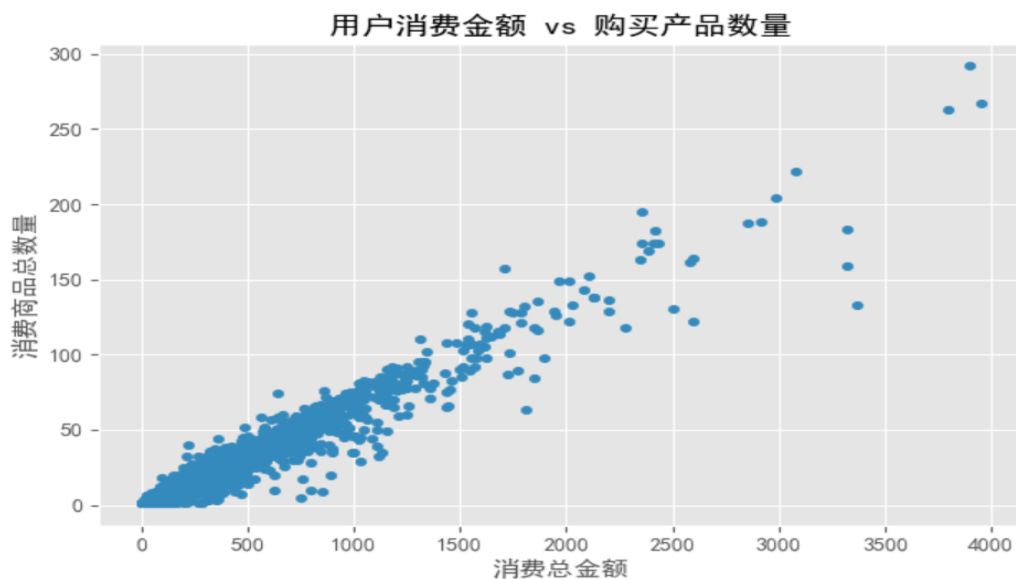
Name: order_count, dtype: float64

(2) 用户消费金额和消费产品数量的散点图

```
# 223428010210 陈梓欣
# 用户消费金额和消费产品数量的散点图

plt.figure(figsize=(8, 5))
user_group.query('order_amount<4000').plot.scatter(x='order_amount',y='order_products')
plt.title('用户消费金额 vs 购买产品数量')
plt.xlabel('消费总金额')
plt.ylabel('消费商品总数量')
plt.grid(True)
plt.tight_layout()
plt.show()
```

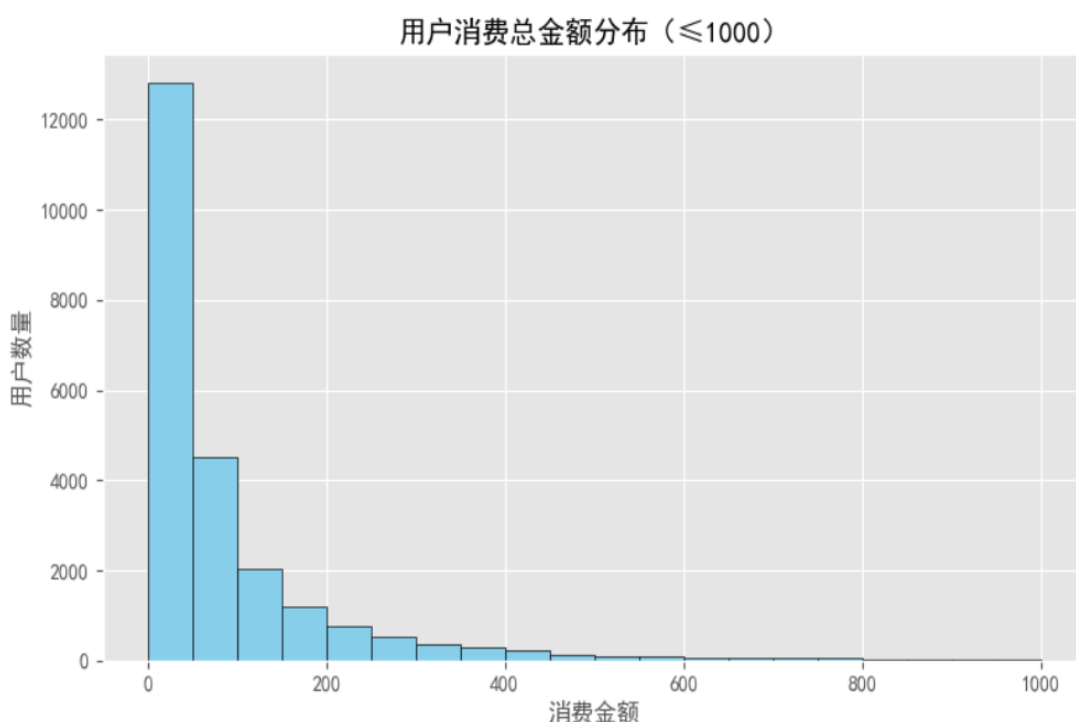
<Figure size 800x500 with 0 Axes>



(3) 各个用户消费总金额的直方分布图(消费金额在 1000 之内的分布)

```
# 223428010210 陈梓欣
# 筛选出消费金额 ≤ 1000 的用户
amount_within_1000 = user_group[user_group['order_amount'] <= 1000]

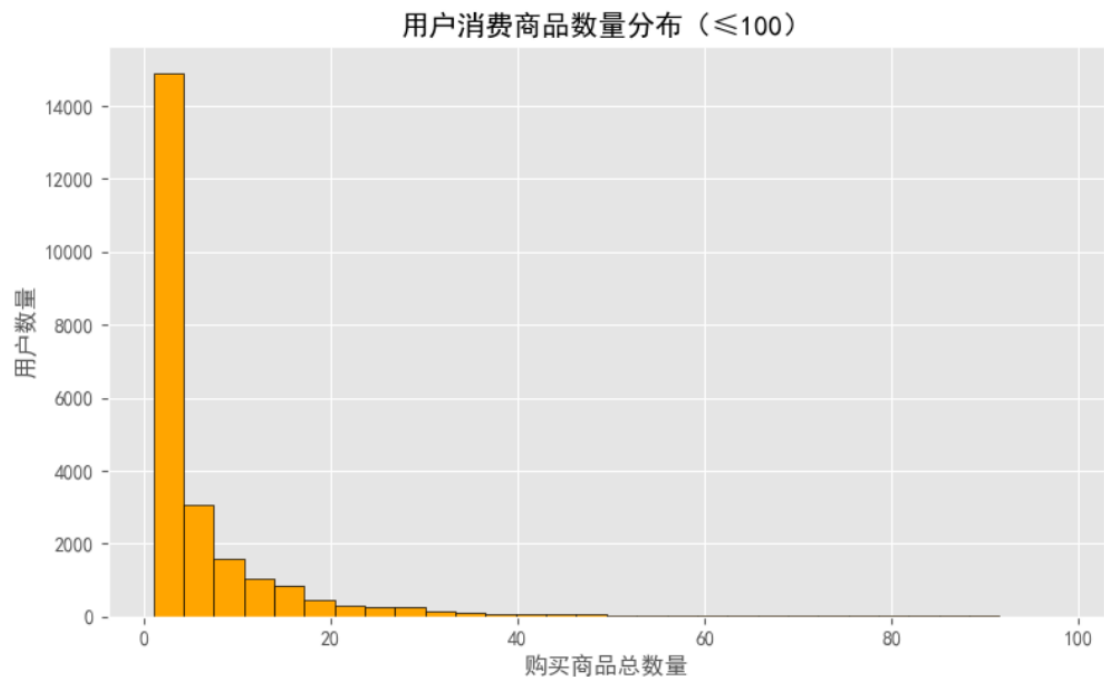
plt.figure(figsize=(8, 5))
plt.hist(amount_within_1000['order_amount'], bins=20, color='skyblue', edgecolor='black')
plt.title('用户消费总金额分布 (≤1000)')
plt.xlabel('消费金额')
plt.ylabel('用户数量')
plt.grid(True)
plt.tight_layout()
plt.show()
```



(4) 各个用户消费的总数量的直方分布图(消费商品的数量在 100 次之内的分布);

```
# 223428010210 陈梓欣
# 筛选出消费数量 ≤ 100 的用户
products_within_100 = user_group[user_group['order_products'] <= 100]

plt.figure(figsize=(8, 5))
plt.hist(products_within_100['order_products'], bins=30, color='orange', edgecolor='black')
plt.title('用户消费商品数量分布 (≤100)')
plt.xlabel('购买商品总数量')
plt.ylabel('用户数量')
plt.grid(True)
plt.tight_layout()
plt.show()
```

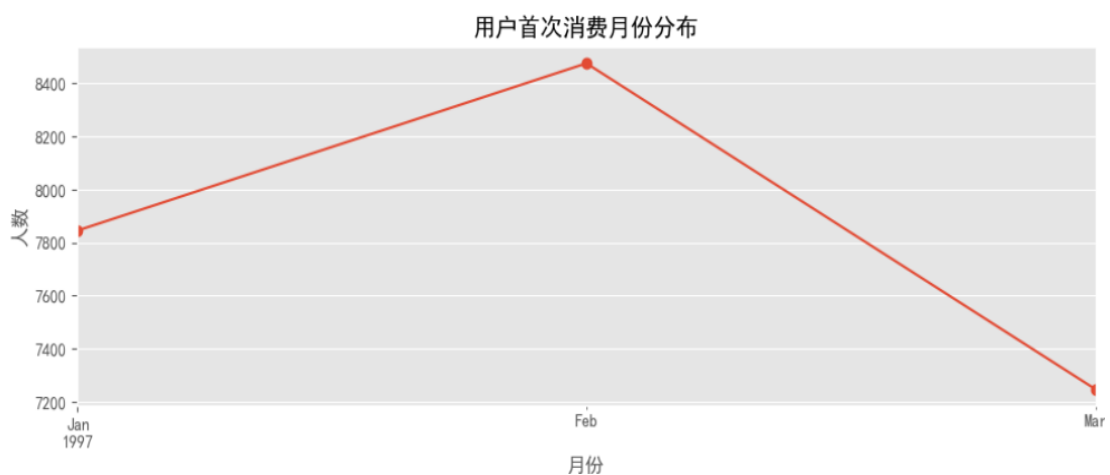


4 用户消费行为分析

(1) 用户第一次消费的月份分布，和人数统计，绘制线形图

```
# 223428010210 陈梓欣
# 获取每个用户的首次消费时间
first_purchase = df.groupby('user_id')['order_dt'].min()
# 转换为月份周期格式
first_purchase_month = first_purchase.dt.to_period('M').value_counts().sort_index()

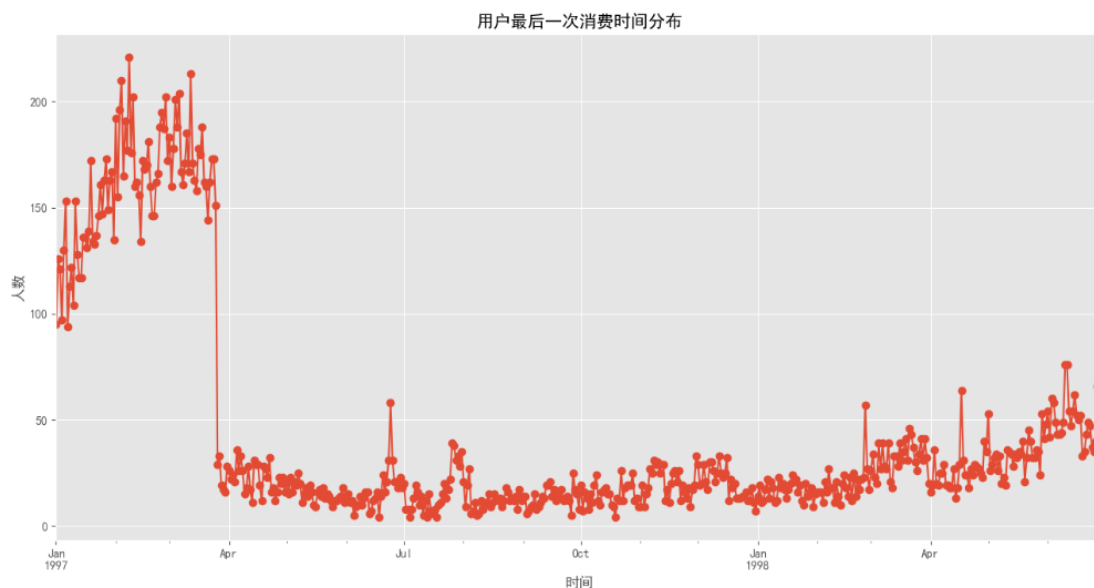
# 绘图
first_purchase_month.plot(title='用户首次消费月份分布', marker='o', figsize=(10, 4))
plt.xlabel('月份')
plt.ylabel('人数')
plt.tight_layout()
plt.show()
```



(2) 用户最后一次消费的时间分布，和人数统计，绘制线形图

```
# 223428010210 陈梓欣
# 用户最后一次消费的时间分布
last_purchase = df.groupby('user_id')['order_dt'].max().value_counts().sort_index()

last_purchase.plot(title='用户最后一次消费时间分布', marker='o', figsize=(13, 7))
plt.xlabel('时间')
plt.ylabel('人数')
plt.grid(True)
plt.tight_layout()
plt.show()
```



(3) 新老客户的占比 消费一次为新用户 消费多次为老用户 分析出每一个用户的第一个消费和最后一次消费的时间 `agg(['func1','func2'])`:对分组后的结果进行指定聚合 分析出新老客户的消费比例

```
# 223428010210 陈梓欣
# 消费次数
user_order_count = df.groupby('user_id').size()

# 消费一次为新用户，多次为老用户
is_new_user = user_order_count == 1
new_ratio = is_new_user.sum() / len(user_order_count)

print(f"新用户比例: {new_ratio:.2%}")
print(f"老用户比例: {(1 - new_ratio):.2%}")
```

新用户比例: 50.52%
老用户比例: 49.48%

(4) 用户分层 分析得出每个用户的总购买量和总消费金额 and 最近一次消费的时间的表格 `rfm` RFM 模型设计 R 表示客户最近一次交易时间的间隔。
`/np.timedelta64(1,'D')`: 去除 days F 表示客户购买商品的总数量,F 值越大,表示客

户交易越频繁，反之则表示客户交易不够活跃。M表示客户交易的金额。M值越大，表示客户价值越高，反之则表示客户价值越低。将R，F，M作用到rfm表中 根据价值分层，将用户分为：重要价值客户 重要保持客户 重要挽留客户 重要发展客户 一般价值客户 一般保持客户 一般挽留客户 一般发展客户 使用已有的分层模型即可 rfm_funcLogistic 回归中终止策略与目标函数值优化

```
# 223428010210 陈梓欣
rfm=df.pivot_table(['order_dt','order_products','order_amount'],index='user_id',
                    aggfunc={'order_dt':'max','order_products':'count','order_amount':'sum'})
```

```
# 223428010210 陈梓欣
rfm['time']=(rfm['order_dt'].max()-rfm['order_dt'])/ np.timedelta64(1, 'D')
print(rfm)
```

	order_amount	order_dt	order_products	time
user_id				
1	11.77	1997-01-01	1	545.0
2	89.00	1997-01-12	2	534.0
3	156.46	1998-05-28	6	33.0
4	100.50	1997-12-12	4	200.0
5	385.61	1998-01-03	11	178.0
...	...	...	...	...
23566	36.00	1997-03-25	1	462.0
23567	20.97	1997-03-25	1	462.0
23568	121.70	1997-04-22	3	434.0
23569	25.74	1997-03-25	1	462.0
23570	94.08	1997-03-26	2	461.0

[23570 rows x 4 columns]

```
# 223428010210 陈梓欣
rfm['time']=(rfm['order_dt'].max()-rfm['order_dt'])/ np.timedelta64(1, 'D')
print(rfm.head())
```

	order_amount	order_dt	order_products	time
user_id				
1	11.77	1997-01-01	1	545.0
2	89.00	1997-01-12	2	534.0
3	156.46	1998-05-28	6	33.0
4	100.50	1997-12-12	4	200.0
5	385.61	1998-01-03	11	178.0

```
# 223428010210 陈梓欣
# 比较每一项和均值的大小，分出高低（实际工作中RFM的划分标准应该以业务为准）
rfm['r_m']=rfm['time']-rfm['time'].mean()
rfm['f_m']=rfm['order_products']-rfm['order_products'].mean()
rfm['m_m']=rfm['order_amount']-rfm['order_amount'].mean()
```

```
def hanshu(x):
    if x>0:
        return 1
    else:
        return 0

rfm['r']=rfm['r_m'].apply(hanshu)
rfm['f']=rfm['f_m'].apply(hanshu)
rfm['m']=rfm['m_m'].apply(hanshu)

print(rfm)
```

	order_amount	order_dt	order_products	time	r_m	f_m \
user_id						
1	11.77	1997-01-01	1	545.0	177.778362	-1.955409
2	89.00	1997-01-12	2	534.0	166.778362	-0.955409
3	156.46	1998-05-28	6	33.0	-334.221638	3.044591
4	100.50	1997-12-12	4	200.0	-167.221638	1.044591
5	385.61	1998-01-03	11	178.0	-189.221638	8.044591
...	...	...	...	...	...	...
23566	36.00	1997-03-25	1	462.0	94.778362	-1.955409
23567	20.97	1997-03-25	1	462.0	94.778362	-1.955409
23568	121.70	1997-04-22	3	434.0	66.778362	0.044591
23569	25.74	1997-03-25	1	462.0	94.778362	-1.955409
23570	94.08	1997-03-26	2	461.0	93.778362	-0.955409

	m_m	r	f	m
user_id				
1	-94.310426	1	0	0
2	-17.080426	1	0	0
3	50.379574	0	1	1
4	-5.580426	0	1	0
5	279.529574	0	1	1
...	...	...	...	...
23566	-70.080426	1	0	0
23567	-85.110426	1	0	0
23568	15.619574	1	1	1
23569	-80.340426	1	0	0
23570	-12.000426	1	0	0

[23570 rows x 10 columns]

```
# 223428010210 陈梓欣
# 比较每一项和均值的大小，分出高低（实际工作中RFM的划分标准应该以业务为准）
rfm['r_m']=rfm['time']-rfm['time'].mean()
rfm['f_m']=rfm['order_products']-rfm['order_products'].mean()
rfm['m_m']=rfm['order_amount']-rfm['order_amount'].mean()

def hanshu(x):
    if x>0:
        return 1
    else:
        return 0

rfm['r']=rfm['r_m'].apply(hanshu)
rfm['f']=rfm['f_m'].apply(hanshu)
rfm['m']=rfm['m_m'].apply(hanshu)

print(rfm.head())
```

	order_amount	order_dt	order_products	time	r_m	f_m \
user_id						
1	11.77	1997-01-01	1	545.0	177.778362	-1.955409
2	89.00	1997-01-12	2	534.0	166.778362	-0.955409
3	156.46	1998-05-28	6	33.0	-334.221638	3.044591
4	100.50	1997-12-12	4	200.0	-167.221638	1.044591
5	385.61	1998-01-03	11	178.0	-189.221638	8.044591

	m_m	r	f	m
user_id				
1	-94.310426	1	0	0
2	-17.080426	1	0	0
3	50.379574	0	1	1
4	-5.580426	0	1	0
5	279.529574	0	1	1

```
# 223428010210 陈梓欣
rfm['jieguo']=rfm['r'].map(str)+rfm['f'].map(str)+rfm['m'].map(str)

v ppp={'111':'重要价值客户',
      '011':'重要保持客户',
      '101':'重要发展客户',
      '001':'重要挽留客户',
      '110':'一般价值客户',
      '010':'一般保持客户',
      '100':'一般发展客户',
      '000':'一般挽留客户'}

rfm['leixing']=rfm['jieguo'].map(ppp)
print(rfm.head())
```

	order_amount	order_dt	order_products	time	r_m	f_m	\
user_id							
1	11.77	1997-01-01	1	545.0	177.778362	-1.955409	
2	89.00	1997-01-12	2	534.0	166.778362	-0.955409	
3	156.46	1998-05-28	6	33.0	-334.221638	3.044591	
4	100.50	1997-12-12	4	200.0	-167.221638	1.044591	
5	385.61	1998-01-03	11	178.0	-189.221638	8.044591	

	m_m	r	f	m	jieguo	leixing
user_id						
1	-94.310426	1	0	0	100	一般发展客户
2	-17.080426	1	0	0	100	一般发展客户
3	50.379574	0	1	1	011	重要保持客户
4	-5.580426	0	1	0	010	一般保持客户
5	279.529574	0	1	1	011	重要保持客户

```
# 223428010210 陈梓欣
# 统计不同类型客户的总消费金额、消费次数
asd = rfm.pivot_table(values=['order_amount', 'order_products'], index='leixing', aggfunc='sum')
print(asd)
```

	order_amount	order_products
leixing		
一般价值客户	36200.21	1782
一般保持客户	141127.20	7371
一般发展客户	409272.88	15589
一般挽留客户	75781.48	3064
重要价值客户	103260.14	1950
重要保持客户	1591666.47	38490
重要发展客户	96849.09	877
重要挽留客户	46158.16	536

```
# 223428010210 陈梓欣
#统计各类用户的人数
print(rfm['leixing'].value_counts())
```

leixing	
一般发展客户	13608
重要保持客户	4617
一般保持客户	1974
一般挽留客户	1532
重要发展客户	579
一般价值客户	543
重要价值客户	449
重要挽留客户	268

Name: count, dtype: int64

5 用户的生命周期

(1) 将用户划分为活跃用户和其他用户统计每个用户每个月的消费次数

```
# 223428010210 陈梓欣
# 统计每个用户每月的消费次数 (即订单数)
user_month_order_count = df.groupby(['user_id', 'month']).size().unstack(fill_value=0)
print(user_month_order_count)
```

month	1997-01-01	1997-02-01	1997-03-01	1997-04-01	1997-05-01	\
user_id						
1	1	0	0	0	0	
2	2	0	0	0	0	
3	1	0	1	1	0	
4	2	0	0	0	0	
5	2	1	0	1	1	
...	...	...	...	...	...	
23566	0	0	1	0	0	
23567	0	0	1	0	0	
23568	0	0	1	2	0	
23569	0	0	1	0	0	
23570	0	0	2	0	0	

month	1997-06-01	1997-07-01	1997-08-01	1997-09-01	1997-10-01	\
user_id						
1	0	0	0	0	0	
2	0	0	0	0	0	
3	0	0	0	0	0	
4	0	0	1	0	0	
5	1	1	0	1	0	
...	...	...	...	...	...	
23566	0	0	0	0	0	
23567	0	0	0	0	0	
23568	0	0	0	0	0	
23569	0	0	0	0	0	
23570	0	0	0	0	0	

month	1997-11-01	1997-12-01	1998-01-01	1998-02-01	1998-03-01	\
user_id						
1	0	0	0	0	0	
2	0	0	0	0	0	
3	2	0	0	0	0	
4	0	1	0	0	0	
5	0	2	1	0	0	
...	...	...	...	...	...	
23566	0	0	0	0	0	
23567	0	0	0	0	0	
23568	0	0	0	0	0	
23569	0	0	0	0	0	
23570	0	0	0	0	0	

month	1998-04-01	1998-05-01	1998-06-01
user_id			
1	0	0	0
2	0	0	0
3	0	1	0
4	0	0	0
5	0	0	0
...	...	...	...
23566	0	0	0
23567	0	0	0
23568	0	0	0
23569	0	0	0
23570	0	0	0

[23570 rows x 18 columns]

(2) 统计每个用户每个月是否消费，消费记录为 1 否则记录为 0 知识点：
 DataFrame 的 apply 和 applymap 的区别 applymap:返回 df 将函数做用于
 DataFrame 中的所有元素(elements) apply:返回 Series apply()将一个函数作用于
 DataFrame 中的每个行或者列

```
# 223428010210 陈梓欣
# 是否消费: 有消费记为1, 无消费记为0
user_month_has_purchase = (user_month_order_count > 0).astype(int)
print(user_month_has_purchase)
```

month	1997-01-01	1997-02-01	1997-03-01	1997-04-01	1997-05-01	\
user_id						
1	1	0	0	0	0	
2	1	0	0	0	0	
3	1	0	1	1	0	
4	1	0	0	0	0	
5	1	1	0	1	1	
...	...	...	...	...	...	
23566	0	0	1	0	0	
23567	0	0	1	0	0	
23568	0	0	1	1	0	
23569	0	0	1	0	0	
23570	0	0	1	0	0	

month	1997-06-01	1997-07-01	1997-08-01	1997-09-01	1997-10-01	\
user_id						
1	0	0	0	0	0	
2	0	0	0	0	0	
3	0	0	0	0	0	
4	0	0	1	0	0	
5	1	1	0	1	0	
...	...	...	...	...	...	
23566	0	0	0	0	0	
23567	0	0	0	0	0	
23568	0	0	0	0	0	
23569	0	0	0	0	0	
23570	0	0	0	0	0	

month	1997-11-01	1997-12-01	1998-01-01	1998-02-01	1998-03-01	\
user_id						
1	0	0	0	0	0	
2	0	0	0	0	0	
3	1	0	0	0	0	
4	0	1	0	0	0	
5	0	1	1	0	0	
...	...	...	...	...	...	
23566	0	0	0	0	0	
23567	0	0	0	0	0	
23568	0	0	0	0	0	
23569	0	0	0	0	0	
23570	0	0	0	0	0	

month	1998-04-01	1998-05-01	1998-06-01
user_id			
1	0	0	0
2	0	0	0
3	0	1	0
4	0	0	0
5	0	0	0
...	...	...	...
23566	0	0	0
23567	0	0	0
23568	0	0	0
23569	0	0	0
23570	0	0	0

[23570 rows x 18 columns]

(3) 将用户按照每一个月份分成：**unreg**:观望用户（前两月没买，第三个月才第一次买,则用户前两个月为观望用户）**unactive**:首月购买后，后序月份没有购买则在没有购买的月份中该用户的为非活跃用户 **new**:当前月就进行首次购买的用户在当前月为新用户 **active**:连续月份购买的用户在这些月中为活跃用户 **return**:购买之后间隔 n 月再次购买的第一个月份为该月份的回头客应用三种不同终止策略，进行回归并给出结果；

```
# 223428010210 陈梓欣
def active_status_v2(row):
    status = []
    purchase_list = list(row)
    first_buy_index = -1

    for i in range(len(purchase_list)):
        val = purchase_list[i]

        # 第一次消费
        if val > 0 and first_buy_index == -1:
            first_buy_index = i
            status.append('new')
        elif first_buy_index == -1:
            # 还未开始消费
            status.append('unreg')
        elif val == 0:
            # 已经开始消费了，现在没买
            if 'new' in status or 'active' in status or 'return' in status:
                status.append('unactive')
            else:
                status.append('unreg') # 前两月没买且第3月才买，前面是 unreg
        else:
            # 当前有消费
            if status[-1] == 'unactive':
                status.append('return')
            elif status[-1] in ['new', 'active', 'return']:
                status.append('active')
            else:
                status.append('active') # 兜底逻辑

    # 处理“前两月没买，第三月首次购买” => 把前两个标记为 unreg
    if 'new' in status:
        new_index = status.index('new')
        if new_index >= 2:
            status[0] = 'unreg'
            status[1] = 'unreg'

    return pd.Series(status)

sdff=dff.apply(active_status,axis=1)
print(sdff)
```

[illegible]

```

...      ...      ...      ...      ...      ...      ...
23566    unreg    unreg    unreg    unreg    unreg    unreg    unreg
23567    unreg    unreg    unreg    unreg    unreg    unreg    unreg
23568    unreg    unreg    unreg    unreg    unreg    unreg    unreg
23569    unreg    unreg    unreg    unreg    unreg    unreg    unreg
23570    unreg    unreg    unreg    unreg    unreg    unreg    unreg

      7      8      9      10     11     12     13  \
user_id
1      unactive unactive unactive unactive unactive unactive unactive
2      unreg    unreg    unreg    unreg    new    unactive unactive
3      unactive unactive unactive unactive unactive unactive unactive
4      unactive unactive unactive unactive unactive unactive unactive
5      unactive unactive unactive unactive unactive unactive    return
...      ...      ...      ...      ...      ...      ...
23566    unreg    unreg    unreg    unreg    unreg    unreg    unreg
23567    unreg    unreg    unreg    unreg    unreg    unreg    unreg
23568    unreg    unreg    unreg    unreg    unreg    unreg    unreg
23569    unreg    unreg    unreg    unreg    unreg    unreg    unreg
23570    unreg    unreg    unreg    unreg    unreg    unreg    unreg

      14     15     16     17
user_id
1      unactive unactive unactive unactive
2      unactive unactive unactive unactive
3      unactive unactive unactive unactive
4      unactive unactive unactive    return
5      unactive unactive unactive unactive
...      ...      ...      ...      ...
23566    unreg    unreg    unreg    unreg
23567    unreg    unreg    unreg    unreg
23568    unreg    unreg    unreg    unreg
23569    unreg    unreg    unreg    unreg
23570    unreg    unreg    unreg    unreg

```

[23570 rows x 18 columns]

五、实验中遇到的问题及解决方法

问题：绘制用户消费金额和消费产品数量的散点图时，数据分布较为集中

解决方法：

绝大部分的数据集中分布，小部分极值对分析有一定的干扰。可以使用 `query` 方法，筛选出 `order_amount<4000` 的用户，排除极值的干扰。

代码为：`mg.query('order_amount<4000').plot.scatter(x='order_amount',y='order_products')dff`。

问题：对用户生命周期状态判断时，`Series` 使用位置索引报未来版本兼容性警告。

解决方法：将 `data[i]` 替换为 `data.iloc[i]`，明确按位置访问，避免版本更新后的索引方式变更问题。

六、实验总结及心得

在本次实验中，我们围绕用户消费行为数据，深入开展了数据加载、清洗、统计分析与可视化等一系列操作。首先通过 `pandas` 读取原始交易数据，并完成了字段命名、缺失值检查、时间格式转换等预处理步骤，为后续分析打下基础。

接着，使用 `groupby` 结合 `sum`、`count` 等函数对用户的消费金额、消费次数及商品数量等关键指标进行了统计，并生成描述性统计信息，帮助理解整体消费行为特征。

在分析过程中，我们还利用 `.groupby().size().unstack()` 技巧构建了“用户-月份”二维消费矩阵，并进一步转化为 0-1 形式，判断用户在每个月是否有消费行为。这一结果为后续生命周期划分（如新用户、活跃用户、流失用户、回流用户等）提供了数据基础。实验中还使用了 `.apply()` 和 `.applymap()` 等函数，明确了它们在行/列与元素级操作中的区别，提升了对 `pandas` 数据操作的理解。

通过本次实验，不仅熟悉了 `Python` 数据分析的流程和方法，还掌握了用户行为分析的关键模型和技术框架，增强了对用户价值评估和精细化运营的理解，具备了初步的数据驱动分析能力，为今后开展大数据分析奠定了良好基础。